National University
Of Computer and Emerging Sciences

Homework #1

Presented to

Sir, Owaids Idrees

In partial fulfillment

of the requirements for the course of

Agentic AI (CS - 4015)

By:

Hafsa Imtiaz

22i - 0959

Section A

AI Research Assistant - Semantic Search Module

The AI Research Assistant uses semantic search to retrieve academic documents based on their meaning, rather than relying only on exact keyword matches. This allows the system to return more relevant and context-aware results.

The system is built in Python and consists of four main components:

1. **Document Processing**
2. **Embedding Generation**
3. **Vector Storage**
4. **Web-Based Interface**

It uses the following technologies:

Component	Technology
Frontend	Streamlit (web-based GUI)
Document Handling	LangChain
Embeddings	HuggingFace Transformers (6 model options)
Vector Storage	FAISS (in-memory) & Chroma (persistent)

System Architecture

The system follows a **modular pipeline architecture**, where each component works independently but connects in a sequential flow. This makes the system easier to maintain, test, and extend in the future.

The data flows in this order:

Document Upload → Text Processing → Embedding Generation → Vector Storage → Similarity Search → Results Display

1. **Data Module (loader.py):**

Loads documents from file uploads or filesystem, performs text chunking with configurable overlap (default: 500 characters, 50 overlap to preserve context across chunk boundary), and converts raw text to LangChain Document objects with metadata preservation.

2. **Embeddings Module (embedding_manager.py):**

Generates semantic vectors using HuggingFace models with automatic GPU/CPU detection. Implements dual-backend architecture (LangChain HuggingFaceEmbeddings with

sentence-transformers fallback) and batch processing for memory efficiency. Supports six models (options defined in config.py):

a. all-MiniLM-L6-v2 *(lightweight, recommended)*
b. all-mpnet-base-v2 *(larger, more accurate)*
c. bge-base-en-v1.5 *(domain-specific)*
d. e5-large-v2
e. e5-small-v2
f. allenai-specter *(academic papers)*

3. **Vector Store Module (store_manager.py):**

Provides two implementations via Factory Pattern: FAISS for fast in-memory search and Chroma for persistent storage. Handles index creation, configurable similarity metrics, and top-k retrieval with metadata.

4. **GUI Module (gui.py):**

Streamlit-based interface with file upload (minimum 10 documents), model/store selection, document statistics, query input, and ranked results displayed with relevance scores.

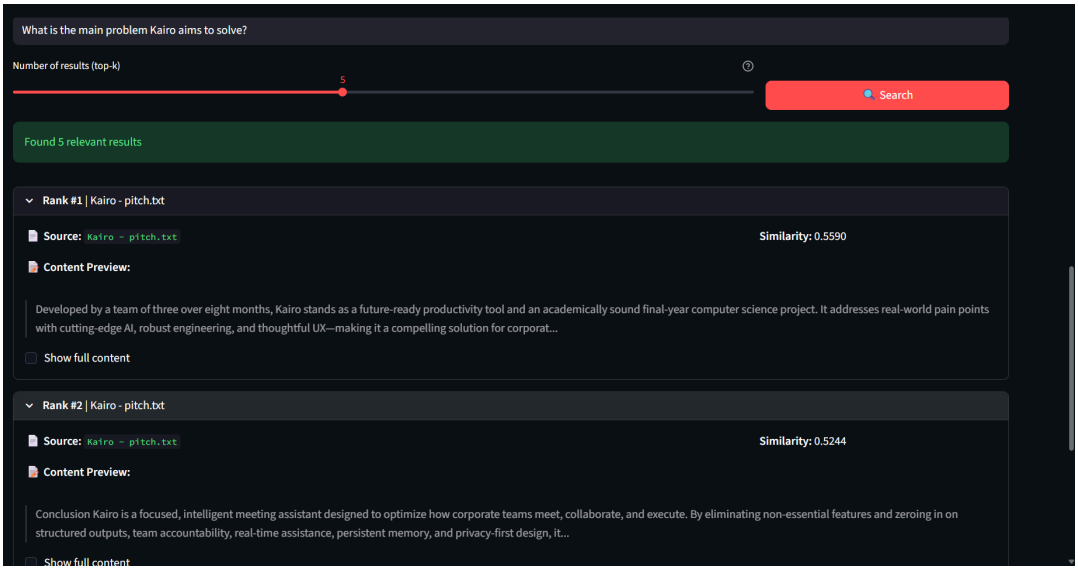
Processing Pipeline

The system processes queries through seven stages:

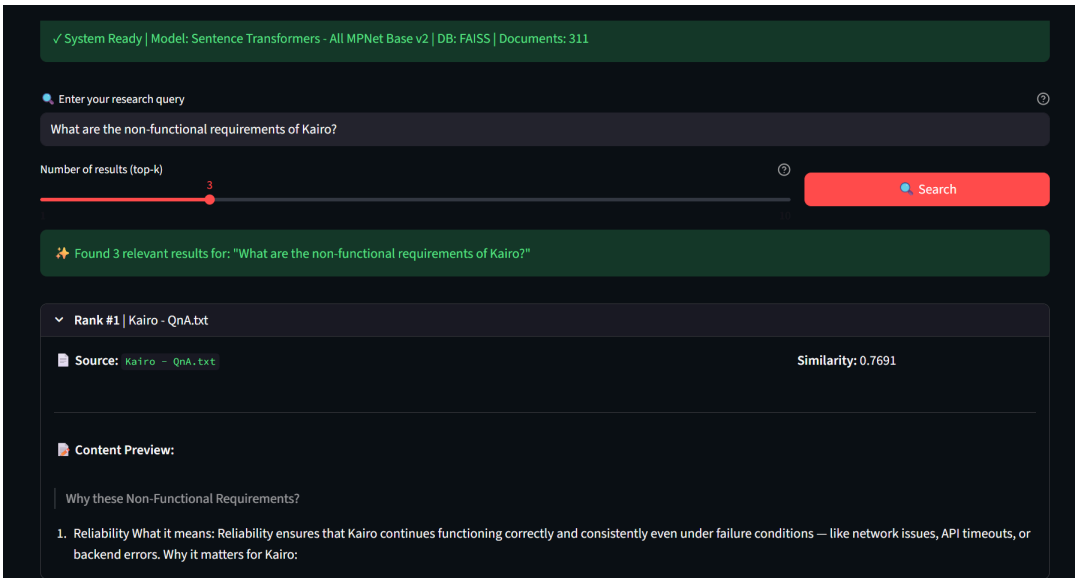
1. Document upload via GUI
2. Text chunking with overlap
3. User selection of embedding model and vector store
4. Batch embedding generation
5. Vector index creation with metadata
6. Query embedding using the same model
7. Similarity search returning top-k ranked results.

Testing & Observations

The system was tested with a custom dataset of 10 documents, containing technical and non-technical research for a Final-Year-Project. The following screenshot is a result of the *all-MiniLM-L6-v2 x FAISS* configuration:



The following screenshot is a result of the *sentence-transformers/all-mpnet-base-v2 x FAISS* configuration:



During testing, the **all-mpnet-base-v2** model with FAISS achieved a maximum similarity score of about **85%** for relevant queries. The results showed that the system correctly understood the meaning of the queries and returned contextually related documents.

Similarity scores did not reach 100%, which is normal in semantic search systems because they measure meaning, not exact word matches. Overall, the system performed well and provided reliable results for academic document retrieval.